

## Table of Contents

### LangChain Interview Guide (Practical, Layman-Terms Edition)

Written the way a 6-7 YOE Agentic AI developer would actually get quizzed — concept first in plain English, then the code an interviewer expects you to write on a whiteboard or shared editor.

---

#### SECTION 1: Fundamentals (they check if you actually understand the “why”)

##### Q1. What problem does LangChain actually solve? Why not just call the OpenAI API directly?

**Layman answer:** Calling an LLM API directly is like talking to a very smart person who has amnesia after every sentence — no memory, no tools, no way to look things up. LangChain is the “scaffolding” around the LLM: it wires up memory, lets the model call tools (search, calculators, databases), chains multiple LLM calls together, and standardizes how you swap one model for another (OpenAI → Anthropic → local model) without rewriting your app.

**What they’re really testing:** whether you’ll reach for LangChain even when a raw API call would’ve been simpler (red flag) or whether you know when it earns its complexity.

*# Without LangChain – works, but doesn't scale past a toy demo*

```
import openai
response = openai.chat.completions.create(
    model="gpt-4",
    messages=[{"role": "user", "content": "Summarize this contract"}]
)
```

*# With LangChain – same call, but now swappable/composable*

```
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-4")
response = llm.invoke("Summarize this contract")
```

---

##### Q2. Explain Chains vs Agents in one sentence each.

**Layman answer:** - **Chain** = a fixed recipe. Step 1 always happens, then step 2, then step 3. No decision-making. - **Agent** = a recipe where the LLM decides which steps to take and in what order, based on the situation — like a chef improvising instead of following a printed recipe.

*# Chain: fixed pipeline – summarize, then translate. Always in that order.*

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
```

```

llm = ChatOpenAI(model="gpt-4")
summarize = ChatPromptTemplate.from_template("Summarize: {text}") | llm
translate = ChatPromptTemplate.from_template("Translate to French: {summary}") | llm

# Agent: LLM decides whether it even needs a tool, and which one
from langchain.agents import create_react_agent, AgentExecutor
# agent decides: "should I search the web, use a calculator, or just answer?"

```

---

### Q3. What is a Prompt Template and why not just use an f-string?

**Layman answer:** An f-string works until you need validation, reuse across 10 different chains, or want to swap in few-shot examples dynamically. PromptTemplate is a structured, reusable, type-checked way to build prompts — like using a form instead of hand-writing a letter every time.

```

from langchain_core.prompts import PromptTemplate

template = PromptTemplate.from_template(
    "You are a {role}. Answer the customer's question: {question}"
)
prompt = template.invoke({"role": "billing support agent", "question":
    "Why was I charged twice?"})

```

---

## SECTION 2: Practical Coding (hands-on, they want to see you type)

### Q4. Build a simple LCEL chain and explain what the | pipe actually does.

**Layman answer:** The | operator is just Unix-pipe style plumbing — output of the left thing becomes input of the right thing. It's LangChain's way of saying "connect these Lego blocks."

```

from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template("Explain {topic} to a 10-year-old")
llm = ChatOpenAI(model="gpt-4")
parser = StrOutputParser()

chain = prompt | llm | parser # prompt text -> LLM -> plain string out
result = chain.invoke({"topic": "blockchain"})

```

**Follow-up they'll ask:** “What if I want structured JSON instead of a string?” – swap `StrOutputParser` for `PydanticOutputParser` or use `.with_structured_output()`.

```
from pydantic import BaseModel

class Answer(BaseModel):
    explanation: str
    difficulty_level: str

structured_llm = llm.with_structured_output(Answer)
chain = prompt | structured_llm
result = chain.invoke({"topic": "blockchain"}) # returns an Answer
object, not raw text
```

---

#### Q5. How does memory work in a chatbot built with LangChain?

**Layman answer:** By default, every LLM call is a blank slate. “Memory” just means: before sending the new question to the model, you paste in the previous conversation as context. LangChain gives you helper classes so you don’t manually manage that list yourself.

```
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

store = {}

def get_history(session_id):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

chain_with_memory = RunnableWithMessageHistory(chain, get_history)

chain_with_memory.invoke(
    {"topic": "blockchain"},
    config={"configurable": {"session_id": "user_123"}}
)
```

**Senior-level follow-up:** “What happens when the conversation gets too long for the context window?” – mention summarization memory (compress old turns into a running summary) or windowed memory (keep only last N turns) instead of storing everything verbatim forever.

---

#### Q6. Write a custom Tool and wire it into an agent.

**Layman answer:** A Tool is just a Python function with a name and description — the LLM reads the description to decide “do I need this?” then calls it like a function.

```

from langchain_core.tools import tool

@tool
def get_order_status(order_id: str) -> str:
    """Look up the shipping status of a customer order by its ID."""
    # pretend this hits a real database
    fake_db = {"1234": "Shipped, arriving Tuesday"}
    return fake_db.get(order_id, "Order not found")

from langchain.agents import create_react_agent, AgentExecutor
from langchain import hub

prompt = hub.pull("hwchase17/react")
agent = create_react_agent(llm, [get_order_status], prompt)
executor = AgentExecutor(agent=agent, tools=[get_order_status],
verbose=True)

executor.invoke({"input": "Where is my order 1234?"})
# Agent reasons: "I should use get_order_status" -> calls it ->
# answers naturally

```

**What they're checking:** does your tool have a clear docstring? A vague description is the #1 reason agents pick the wrong tool in production.

---

## SECTION 3: RAG & Retrieval (almost guaranteed in any agentic AI interview)

### Q7. Explain RAG in one paragraph, no jargon.

**Layman answer:** The LLM doesn't know your company's private documents. RAG is: (1) chop your documents into chunks, (2) convert each chunk into a number-vector ("embedding") that captures its meaning, (3) store those in a searchable vector database, (4) when a user asks a question, find the most similar chunks and paste them into the prompt as context, (5) ask the LLM to answer using only that pasted context. It's "open-book exam" instead of "answer from memory."

```

from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

# 1. Load
docs = TextLoader("company_policy.txt").load()

# 2. Split into chunks (overlap keeps context from being cut mid-
sentence)
splitter = RecursiveCharacterTextSplitter(chunk_size=500,
chunk_overlap=50)
chunks = splitter.split_documents(docs)

```

```

# 3. Embed + store
vectorstore = FAISS.from_documents(chunks, OpenAIEmbeddings())

# 4. Retrieve + 5. Answer
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})

from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough

rag_prompt = ChatPromptTemplate.from_template(
    "Answer using ONLY this context:\n{context}\n\nQuestion: {question}"
)

def format_docs(docs):
    return "\n\n".join(d.page_content for d in docs)

rag_chain = (
    {"context": retriever | format_docs, "question":
    RunnablePassthrough()}
    | rag_prompt | llm | StrOutputParser()
)

rag_chain.invoke("What is our remote work policy?")

```

---

**Q8. Your RAG bot is confidently making up answers not found in the documents. How do you fix it?**

**Layman answer:** This is the classic “hallucination on top of retrieval” problem. The fix isn’t one thing — it’s a checklist: 1. **Tighten the prompt** — explicitly instruct “if the answer isn’t in the context, say you don’t know.” 2. **Check retrieval quality first** — often the real bug is that the retriever pulled irrelevant chunks, so the LLM is just working with garbage. 3. **Add a relevance/grounding check** — a second LLM call or a similarity-score threshold that rejects low-confidence retrievals. 4. **Tune chunk size** — chunks too small lose context, too large dilute relevance. 5. **Use metadata filtering** — narrow the search space (e.g., only search “HR” documents for HR questions) before doing similarity search.

```

rag_prompt = ChatPromptTemplate.from_template(
    "Answer using ONLY this context. If the answer is not present, "
    "say 'I don't have that information' – do not guess.\n\n"
    "Context:\n{context}\n\nQuestion: {question}"
)

```

```

# Score-based filtering to drop weak matches before they reach the
prompt
results_with_scores = vectorstore.similarity_search_with_score(query,
k=5)

```

```
good_matches = [doc for doc, score in results_with_scores if score < 0.3] # lower = more similar for L2
```

---

#### Q9. Similarity search vs MMR (Maximal Marginal Relevance) — when would you use each?

**Layman answer:** Plain similarity search returns the top-K most similar chunks — but if your document repeats the same idea five times, you'll get five near-duplicate chunks, wasting context space. MMR deliberately picks chunks that are relevant AND different from each other, like a news editor picking diverse sources instead of five articles saying the same thing.

```
retriever = vectorstore.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 4, "fetch_k": 20, "lambda_mult": 0.5}
    # lambda_mult: 1 = pure relevance, 0 = pure diversity
)
```

---

### SECTION 4: Production & Architecture (this is where 6-7 YOE gets separated from junior devs)

Q10. Your agent works fine in testing but is slow and expensive in production. Walk me through your debugging approach.

**Layman answer:** Treat it like any performance problem — measure before you guess. 1. **Trace it** — use LangSmith (or custom logging) to see exactly how many LLM calls happen per user request. Agents often make 5-10 calls when 2 would do. 2. **Cache** — identical or near-identical queries shouldn't hit the LLM twice. 3. **Right-size the model** — don't use GPT-4-class models for tasks a cheaper/smaller model can handle (e.g., intent classification). 4. **Reduce retrieved context** — sending 20 chunks when 4 would answer the question burns tokens for nothing. 5. **Stream responses** — doesn't reduce cost, but massively improves perceived latency.

```
from langchain.globals import set_llm_cache
from langchain_community.cache import InMemoryCache
```

```
set_llm_cache(InMemoryCache()) # identical prompts skip the API call entirely
```

```
# Streaming for perceived speed
for chunk in chain.stream({"topic": "blockchain"}):
    print(chunk, end="", flush=True)
```

---

### Q11. How do you handle an LLM call failing (timeout, rate limit, malformed output) in production?

**Layman answer:** Never trust a single LLM call to just work. Wrap it the same way you'd wrap any flaky network call — retries with backoff, fallback models, and output validation.

```
from langchain_openai import ChatOpenAI
from langchain_core.runnables import RunnableRetry

llm = ChatOpenAI(model="gpt-4").with_retry(
    stop_after_attempt=3,
    wait_exponential_jitter=True
)

# Fallback: if GPT-4 fails entirely, drop to a cheaper backup model
primary = ChatOpenAI(model="gpt-4")
backup = ChatOpenAI(model="gpt-3.5-turbo")
robust_llm = primary.with_fallbacks([backup])
```

---

### Q12. How would you evaluate whether your LangChain app's outputs are actually good, not just "it ran without errors"?

**Layman answer:** "It didn't crash" is not the same as "it gave a correct answer." You need actual evaluation: - **Golden dataset** — a set of question/expected-answer pairs you test against every time you change a prompt or model. - **LLM-as-judge** — use a separate LLM call to grade the output against a rubric (helpful, accurate, grounded in context). - **LangSmith** — Anthropic/LangChain's tracing+eval tool to track this over time instead of eyeballing logs.

```
from langchain.evaluation import load_evaluator

evaluator = load_evaluator("qa") # compares generated answer vs
reference answer
result = evaluator.evaluate_strings(
    input="What is our refund policy?",
    prediction="Refunds are processed within 5 business days.",
    reference="Refunds take 5-7 business days to process."
)
print(result) # {'score': ..., 'reasoning': ...}
```

---

### Q13. LangChain vs LlamaIndex vs just rolling your own — how do you decide?

**Layman answer (give a balanced, non-dogmatic answer — interviewers like this):** -

**LangChain** — best when your app needs agents, tool-calling, and complex multi-step orchestration (LCEL/LangGraph shine here). - **LlamaIndex** — historically stronger for pure RAG/indexing-heavy use cases with more retrieval-strategy options out of the box. - **Roll your own** — makes sense for simple, single-purpose apps where the abstraction overhead of a

framework isn't worth it, or when you need tight control over latency/cost and the framework's abstractions get in the way.

There's no universally "right" answer — a senior engineer picks based on the actual requirements, not framework loyalty.

---

## SECTION 5: Advanced / Agentic (for roles that explicitly say "Agentic AI")

### Q14. What is LangGraph and why would you reach for it instead of a plain AgentExecutor?

**Layman answer:** A standard agent loop (AgentExecutor) is a simple loop: think → act → observe → repeat, until done. That's fine for simple cases, but breaks down when you need: multiple agents cooperating, conditional branching ("if the answer needs approval, route to a human"), loops with explicit state, or the ability to resume/replay a specific step. LangGraph models your agent as a **graph** (nodes = steps, edges = transitions) instead of a rigid loop — like moving from a flowchart with one path to one with actual branches and cycles.

```
from langgraph.graph import StateGraph, END
from typing import TypedDict

class State(TypedDict):
    question: str
    answer: str
    needs_human_review: bool

def answer_node(state: State) -> State:
    state["answer"] = llm.invoke(state["question"]).content
    state["needs_human_review"] = "refund" in
state["question"].lower()
    return state

def route(state: State) -> str:
    return "human_review" if state["needs_human_review"] else END

graph = StateGraph(State)
graph.add_node("answer", answer_node)
graph.set_entry_point("answer")
graph.add_conditional_edges("answer", route, {"human_review":
"human_review", END: END})
graph.add_node("human_review", lambda s: s) # placeholder node
graph.add_edge("human_review", END)

app = graph.compile()
app.invoke({"question": "Can I get a refund?", "answer": "",
"needs_human_review": False})
```

---



### Q15. How do you stop an agent from going into an infinite tool-calling loop?

**Layman answer:** This happens when the agent keeps thinking “I need more info” and keeps calling tools without converging on an answer. Guardrails: 1. **Max iterations** — hard cap on how many loop cycles it can run. 2. **Max execution time** — a wall-clock timeout. 3. **Better prompting** — explicitly tell the agent when to stop (“if you have enough information, answer directly — do not call more tools”).

```
executor = AgentExecutor(  
    agent=agent,  
    tools=[get_order_status],  
    max_iterations=5,           # hard stop  
    max_execution_time=30,      # seconds  
    handle_parsing_errors=True # don't crash on a malformed agent  
response  
)
```

---

### Q16. How would you design a multi-agent system where one agent does research and another writes the final report?

**Layman answer:** This is basically an assembly line — one specialist gathers raw material, hands it off, another specialist turns it into a polished product. In LangGraph, that’s two nodes with a shared state object passed between them.

```
class ReportState(TypedDict):  
    topic: str  
    research_notes: str  
    final_report: str  
  
def researcher_node(state: ReportState) -> ReportState:  
    state["research_notes"] = llm.invoke(  
        f"Research key facts about: {state['topic']}"  
    ).content  
    return state  
  
def writer_node(state: ReportState) -> ReportState:  
    state["final_report"] = llm.invoke(  
        f"Write a polished report using these notes:\n{state['research_notes']}"  
    ).content  
    return state  
  
graph = StateGraph(ReportState)  
graph.add_node("researcher", researcher_node)  
graph.add_node("writer", writer_node)  
graph.set_entry_point("researcher")  
graph.add_edge("researcher", "writer")  
graph.add_edge("writer", END)
```

```
app = graph.compile()
result = app.invoke({"topic": "EV battery trends 2026",
"research_notes": "", "final_report": ""})
```

**Follow-up they love asking:** “What if the two agents disagree or the writer needs to send it back for more research?” — mention conditional edges that route back to the researcher node (a loop), with a max-iteration guard so it doesn’t spiral.

---

## Quick-Fire Round (rapid answers they might throw at you)

| Question                                                                                           | One-line answer                                                                                                                                                                 |
|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| What’s the difference between <code>invoke</code> , <code>batch</code> , and <code>stream</code> ? | <code>invoke</code> = one input, one output. <code>batch</code> = many inputs, parallelized. <code>stream</code> = tokens returned as generated, not all at once.               |
| What is <code>RunnablePassthrough</code> ?                                                         | Lets a value flow through unchanged into a later step — a “no-op” wire in the pipeline.                                                                                         |
| Why use <code>with_structured_output()</code> instead of parsing JSON yourself?                    | It leans on the model’s native function/tool-calling to guarantee schema-valid output, instead of hoping the model formats JSON correctly.                                      |
| What’s a vector store vs a vector index?                                                           | Vector store = the database (Pinecone, FAISS, Chroma). Index = the internal data structure it uses to search fast (e.g., HNSW).                                                 |
| How do you version-control prompts?                                                                | Treat them like code — LangSmith Hub, or just store them in your repo with tests, not hardcoded inline strings scattered everywhere.                                            |
| What’s the risk of giving an agent too many tools?                                                 | The LLM’s tool-selection accuracy drops as the tool list grows — beyond ~10-15 tools, consider hierarchical agents (a router picks a sub-agent, which picks a smaller toolset). |

## How to use this doc

Read Section 1-2 the night before for warmup, then rehearse Section 3-5 out loud — those are the ones that actually differentiate a senior candidate. The Quick-Fire table is good for a final skim right before the call.